

Release Notes & Change Log Drafter from Git Commits

Complete Project Documentation • Implementation Guide • Development Roadmap

Project purpose: Automatically read Git commits and transform technical commit history into clear, structured release notes and a detailed change log using Python, Git, and an AI model.

1. Project Overview

This project is a developer productivity application that reduces the manual work involved in preparing release notes. The application reads commits from a Git repository, cleans and categorizes the messages, sends useful commit information to an AI model, and generates a reviewable release-notes draft.

A Flask web application is recommended for the implementation. The user can provide a repository path and commit range, generate the notes, edit the result, and export it as Markdown, TXT, or PDF.

2. Problem Statement

Git repositories may contain many commits between releases. Commit messages are often written for developers rather than customers, and messages can be inconsistent, repetitive, or too technical. Preparing release notes manually is time-consuming and can cause important changes to be overlooked.

Goal: Convert raw Git commit history into concise, organized, human-readable release notes and a detailed change log without inventing changes that are not present in the source commits.

3. Proposed Solution

Step	Action	Result
1	Select repository and commit range	Input configuration
2	Read Git history	Structured commit data
3	Clean commit messages	Normalized messages
4	Classify changes	Feature / Fix / Docs / Refactor / Other
5	Generate AI summary	Readable release-note draft
6	Review and edit	Approved content
7	Export	Markdown / TXT / PDF

4. Main Features

- **Git Commit Reader:** Reads hashes, authors, dates, and commit messages.
- **Commit Range Selection:** Supports latest commits, tags, branches, dates, or two commit IDs.
- **Commit Classification:** Groups changes into features, fixes, improvements, documentation, refactoring, tests, performance, and other changes.
- **AI Release Note Generator:** Converts developer-oriented messages into readable release notes.
- **Change Log Generator:** Keeps a more detailed record of individual changes.
- **Noise Filtering:** Can ignore or reduce merge commits and trivial messages.
- **Human Review:** Lets the user edit the generated draft before export.
- **Export:** Saves the final result as Markdown, text, and optionally PDF.

5. Technology Stack

Layer	Technology
Language	Python 3.11+
Web framework	Flask
Git integration	Git CLI or GitPython
AI integration	Groq Python SDK or another supported LLM API
Configuration	python-dotenv
Frontend	HTML, CSS, JavaScript
Export	Markdown / TXT / ReportLab PDF
Version control	Git + GitHub

6. Project Structure

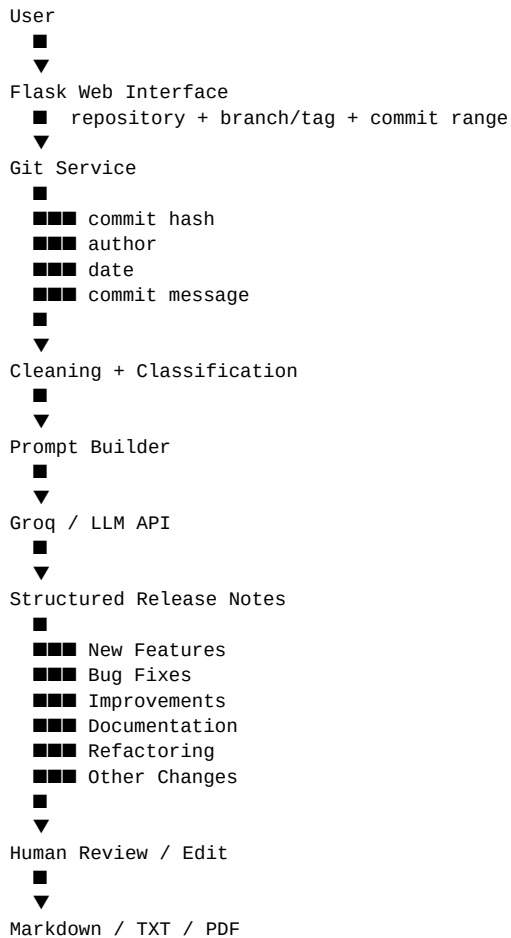
```
release-notes-drafter/  
■  
■■■ app.py  
■■■ requirements.txt  
■■■ .env  
■■■ .gitignore  
■■■ README.md  
■  
■■■ services/  
■   ■■■ __init__.py  
■   ■■■ git_service.py  
■   ■■■ classifier_service.py  
■   ■■■ ai_service.py  
■   ■■■ release_service.py  
■  
■■■ prompts/  
■   ■■■ release_notes_prompt.txt  
■  
■■■ templates/  
■   ■■■ index.html  
■  
■■■ static/  
■   ■■■ css/style.css  
■   ■■■ js/script.js  
■  
■■■ outputs/  
■   ■■■ .gitkeep  
■  
■■■ tests/  
■   ■■■ test_git_service.py  
■   ■■■ test_release_service.py
```

7. File Responsibilities

File	Purpose
app.py	Flask entry point, routes, validation, and response handling.
git_service.py	Reads Git commit history and converts it into structured Python data.
classifier_service.py	Assigns categories to commits.
ai_service.py	Builds the AI request and generates release notes.
release_service.py	Combines, formats, and exports final results.
release_notes_prompt.txt	Reusable AI instructions and output rules.
index.html	Web form and results interface.
script.js	Calls Flask APIs and displays results.

File	Purpose
style.css	Frontend styling.
.env	Stores secret configuration locally.
requirements.txt	Python dependencies.

8. System Architecture and Data Flow



9. Git Commit Extraction

For the first implementation, use the Git command-line tool because it works directly with normal Git repositories. Use a structured separator so commit fields can be parsed safely.

```
git log --date=iso --pretty=format:"%H%x1f%an%x1f%ad%x1f%s%x1e" -n 50
```

Each commit should become a Python dictionary containing `commit_hash`, `author`, `date`, and `message`. Use `subprocess` with an argument list rather than building an unsafe shell command from user input.

10. Commit Classification

Category	Typical indicators
Features	feat, feature, add, introduce, implement
Bug Fixes	fix, bug, error, issue, patch
Improvements	improve, enhance, update
Refactoring	refactor, clean, restructure, rename
Documentation	docs, documentation, README
Tests	test, tests, coverage
Performance	performance, speed, cache, latency
Other	No clear match

11. AI Prompt Design

The AI must summarize only information supported by the supplied commits. The prompt should require concise language, logical grouping, duplicate reduction, and preservation of important technical names when relevant.

You are a professional software release-notes writer.

Given the supplied Git commits:

1. Group related commits.
2. Categorize them as Features, Bug Fixes, Improvements, Documentation, Refactoring, Tests, Performance, or Other.
3. Rewrite technical commit messages into concise user-facing statements.
4. Do not invent functionality or changes.
5. Remove duplicate or trivial noise where appropriate.
6. Return clean Markdown with headings and bullet points.

Also provide a Change Log containing:

- date
- category
- short description
- short commit hash

COMMITTS:
{COMMITTS}

12. Example

Input Git commits:

```
a81d21f Add password reset flow
b77f920 Fix login error on expired sessions
c19ab10 Update README installation steps
d31ce44 Improve dashboard loading time
```

Generated release notes:

```
## Release Notes

### New Features
- Added a password reset flow for account recovery.

### Bug Fixes
- Fixed login failures caused by expired sessions.

### Improvements
- Improved dashboard loading performance.

### Documentation
- Updated the installation instructions in the README.
```

13. Flask API Routes

Method	Route	Purpose
GET	/	Show application interface.
POST	/api/commits	Read commits from repository/range.
POST	/api/generate	Generate release notes.
POST	/api/export	Export reviewed output.

14. Environment Configuration

```
GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=your_supported_groq_model
FLASK_ENV=development
```

Never hard-code the API key and never commit .env to GitHub. Add .env to .gitignore.

15. Installation and Setup

```
# Create project
mkdir release-notes-drafter
```

```

cd release-notes-drafter

# Create virtual environment
python -m venv venv

# Windows PowerShell
venv\Scripts\Activate.ps1

# Install packages
pip install -r requirements.txt

# Configure .env

# Run
python app.py

```

16. Exact Development Order — What You Do Next

Phase	Implementation task	Expected result
1	Create folders, venv, requirements.txt	Project starts cleanly
2	Build <code>git_service.py</code>	Commits are extracted
3	Test Git parser	Correct structured commit data
4	Build <code>classifier_service.py</code>	Commits have categories
5	Build <code>ai_service.py</code>	AI returns a draft
6	Build <code>release_service.py</code>	Formatted release notes
7	Build Flask UI	User can submit inputs
8	Add review/edit screen	User controls final content
9	Add Markdown/TXT/PDF export	Final files are produced
10	Add tests and README	Project is ready to demonstrate

17. Testing Checklist

- Test a repository containing at least 10 commits.
- Test an empty commit range.
- Test invalid repository paths.
- Test invalid Git ranges.
- Test merge commits and duplicate messages.
- Test unusual characters in commit messages.
- Test invalid or unavailable AI API credentials.
- Verify that AI output does not invent changes.
- Verify Markdown, TXT, and PDF export.
- Verify that secrets are absent from Git history.

18. Security and Reliability

- Validate repository paths.
- Use subprocess argument lists instead of unsafe shell-string construction.
- Limit the number of commits sent to the AI model.
- Never expose API keys to browser JavaScript.
- Keep `.env` in `.gitignore`.

- Return safe error messages without exposing secrets.
- Require human review before publishing release notes.

19. Final Project Outcome

When completed, the application will allow a developer to select a Git repository and commit range, automatically extract and classify changes, generate readable release notes with AI, review/edit the result, and export the final document. The project demonstrates Python development, Git integration, GenAI, prompt engineering, API integration, Flask web development, text processing, and document generation.

20. Future Enhancements

- Automatic version detection from Git tags.
- Conventional Commits support.
- GitHub/GitLab repository integration.
- Pull-request based release notes.
- Author/team contribution summaries.
- Release note templates for different audiences.
- Automatic changelog updates.
- Email or Slack delivery of generated release notes.